

[Sistemas Operativos de Código Abierto.]

LABORATORIO N.º 15

Integrantes:

Apellidos y Nombres	Sección.	Grupo.
Martel Vasquez Yazuri	B	

Docente: Godofredo Diaz

Fecha de realización: 24 de junio, 2026

Fecha de entrega: 24 de junio, 2026

Hardware en Linux y entorno gráfico.

I. Capacidades

- Solucionar problemas utilizando los logs generados por el sistema.

II. Seguridad

- En este laboratorio está prohibida la manipulación del hardware, conexiones eléctricas o de red. Así como la ingesta de alimentos y bebidas.
- Ubicar maletines y/o mochilas en lugar destinado para tal fin.
- Dejar la mesa de trabajo y la silla utilizada limpias y ordenadas.

III. Fundamento teórico

- Revise el material de la semana correspondiente antes del desarrollo del laboratorio.

IV. Normas empleadas

- a. Ubicar maletines y/o mochilas en el gabinete al final del aula de laboratorio o en los casilleros asignados al estudiante.
- b. Al culminar la sesión de laboratorio apagar correctamente la computadora y la pantalla, y ordenar las sillas utilizadas.

V. Recursos

- a. Computadora con Vmware Workstation.
- b. ISOS de Ubuntu.
- c. [Bot](#)
- d. [Webserver](#)

VI. Metodología para el desarrollo de la tarea

- El desarrollo del laboratorio es **individual**

VII. Procedimiento

Fuente: <https://github.com/gdiazes/SOCA/blob/main/Semana15/laboratorio15.md>

Contexto del Escenario

Tu servidor web Apache (10.160.10.200) aloja un juego interactivo y una base de datos. Repentinamente, los usuarios reportan que el sistema está inoperativo o extremadamente lento (Jitter/Intermitencia). Tienes confirmación de que un nodo hostil (10.160.10.100) está operando en tu red.

Tu misión es utilizar el método científico (Flujo HP) para aislar la falla, diagnosticar los vectores de ataque y mitigar la amenaza sin desconectar a los usuarios legítimos.

FASE 1: Recopilación de Información (HP Paso 1)

Objetivo: Establecer la línea base y confirmar la anomalía documentando evidencia en todas las capas (Red, Hardware y Aplicación).

Ejecuta estas herramientas por separado (abre varias terminales) y documenta tus hallazgos:

Capa a Analizar	Comando de Diagnóstico	Reto Analítico para tu Reporte
Transferencia de Red	<code>nload ens33</code>	Reto: Lee el manual (<code>nload -h</code>). ¿Qué opciones debes agregar al comando para que el tráfico se muestre en Megabytes (M) y se refresque cada 200ms? Si el TX (Salida) es muy alto, ¿qué significa?
Estado de Sockets	<code>ss -antp grep -E 'ESTAB SYN-RECV'</code>	Si ves cientos de conexiones SYN-RECV, ¿qué te dice esto sobre la naturaleza del tráfico que golpea la pila TCP del Kernel?
Monitoreo de Logs	<code>tail -f /var/log/apache2/access.log</code>	Usa <code>grep "db.sql"</code> sobre este comando. ¿A qué velocidad se registran las descargas? ¿Las peticiones provienen de una única IP o de varias?
Uso de CPU/Memoria	<code>htop</code>	Observa los procesos de <code>apache2</code> . ¿Están consumiendo CPU o Memoria de forma inusual? (Presiona <code>q</code> para salir).
Uso de Disco (I/O)	<code>iostat -x 1</code>	Observa la columna <code>%util</code> . Si el disco está al 100%, ¿es un fallo de hardware o consecuencia de las descargas masivas? (Nota: Requiere <code>sudo apt install sysstat</code>).

Procedimientos:

- **Transferencia de Red:**



Descripción: En la imagen se puede visualizar: Gráfica de paquetes que ingresan y salen del Servidor. Como se observa, se ve una gran proporción de paquetes que salen del sistema. Eso confirma que el atacante ha estado bajando archivos.

Reto: Lee el manual (`nload -h`). ¿Qué opciones debes agregar al comando para que el tráfico se muestre en Megabytes (M) y se refresque cada 200ms? Si el TX (Salida) es muy alto, ¿qué significa?

```
alumno@webserver:~$ nload -h

nload version 0.7.4
Copyright (C) 2001 - 2012 by Roland Riegel <feedback@roland-riegel.de>
nload comes with ABSOLUTELY NO WARRANTY. This is free software, and you are
welcome to redistribute it under certain conditions. For more details see the
GNU General Public License Version 2 (http://www.gnu.org/copyleft/gpl.html).

Command line syntax:
nload [options] [devices]
nload --help|-h

Options:
-a period          Sets the length in seconds of the time window for average
                   calculation.
                   Default is 300.
-i max_scaling     Specifies the 100% mark in kBit/s of the graph indicating the
                   incoming bandwidth usage. Ignored if max_scaling is 0 or the
                   switch -m is given.
                   Default is 10240.
-m               Show multiple devices at a time; no traffic graphs.
-o max_scaling     Same as -i but for the graph indicating the outgoing bandwidth
                   usage.
                   Default is 10240.
-t interval       Determines the refresh interval of the display in milliseconds.
                   Default is 500.
-u h|b|k|m|g      Sets the type of unit used for the display of traffic numbers.
                   h: auto, b: Bit/s, k: kBit/s, m: MBit/s etc.
                   H: auto, B: Byte/s, K: kByte/s, M: MByte/s etc.
                   Default is h.
-U h|b|k|m|g      Same as -u, but for a total amount of data (without "/s").
                   H: auto, B: Byte/s, K: kByte/s, M: MByte/s etc.
                   Default is H.
devices           Network devices to use.
                   Default is to use all auto-detected devices.
--help           Print this help.
-h
```

example: `nload -t 200 -i 1024 -o 128 -U M`

The options above can also be changed at run time by pressing the 'F2' key.

Respuesta: `nload -t 200 -u M`

- **Estado de Sockets:**

```

SYN-RECV 0      0      [::ffff:10.160.10.200]:80      [::ffff:248.227.176.146]:1098
SYN-RECV 0      0      [::ffff:10.160.10.200]:80      [::ffff:246.171.230.150]:2319
SYN-RECV 0      0      [::ffff:10.160.10.200]:80      [::ffff:244.205.15.110]:2399
SYN-RECV 0      0      [::ffff:10.160.10.200]:80      [::ffff:248.151.242.213]:1703
SYN-RECV 0      0      [::ffff:10.160.10.200]:80      [::ffff:255.177.141.153]:2462
SYN-RECV 0      0      [::ffff:10.160.10.200]:80      [::ffff:255.110.249.11]:2415
SYN-RECV 0      0      [::ffff:10.160.10.200]:80      [::ffff:253.126.71.53]:1620
SYN-RECV 0      0      [::ffff:10.160.10.200]:80      [::ffff:249.190.178.107]:2418
SYN-RECV 0      0      [::ffff:10.160.10.200]:80      [::ffff:250.135.114.24]:2801
SYN-RECV 0      0      [::ffff:10.160.10.200]:80      [::ffff:243.163.238.204]:1712
SYN-RECV 0      0      [::ffff:10.160.10.200]:80      [::ffff:249.65.45.65]:2903
SYN-RECV 0      0      [::ffff:10.160.10.200]:80      [::ffff:249.253.83.106]:1913
SYN-RECV 0      0      [::ffff:10.160.10.200]:80      [::ffff:254.233.237.91]:2450
SYN-RECV 0      0      [::ffff:10.160.10.200]:80      [::ffff:253.168.233.201]:2220
SYN-RECV 0      0      [::ffff:10.160.10.200]:80      [::ffff:241.141.108.158]:1542
SYN-RECV 0      0      [::ffff:10.160.10.200]:80      [::ffff:243.189.21.153]:2063
SYN-RECV 0      0      [::ffff:10.160.10.200]:80      [::ffff:241.60.250.248]:2726
SYN-RECV 0      0      [::ffff:10.160.10.200]:80      [::ffff:240.85.83.115]:1339
alumno@webserver:~$

```

Descripción: Se observa en la imagen el Historial de Logs que se han registrado durante el estado activo del servidor. Se visualiza la fecha, mes, y hora exacta que se registró los últimos movimientos o notificaciones de acción que se tomo: En este caso se ve que el atacante logró una comunicación con el servicio Web (HTTP) del servidor y logró obtener archivos del sistema `/db.sql` con `curl/8.18.0`

Reto: Usa `grep "db.sql"` sobre este comando. ¿A qué velocidad se registran las descargas? ¿Las peticiones provienen de una única IP o de varias?

Respuesta: Al observar el log filtrado, se evidencia que las peticiones se registran a una velocidad altísima (múltiples descargas de `db.sql` en el mismo segundo: 14:22:54 y 14:22:56). Además, todas estas peticiones provienen de una única dirección IP hostil (10.160.10.100) utilizando la herramienta `curl/8.18.0`

- **Uso de CPU/Memoria:**

```

0[|||
1[|||
Mem[|||||||||||||||||
Swp[|||||

2.0%] Tasks: 37, 84 thr, 193 kthr; 1 running
1.3%] Load average: 0.08 0.03 0.01
345M/3.27G] Uptime: 01:00:38
0K/3.77G]

Main I/O
PID USER PRI NI VIRT RES SHR S CPU% MEM% TIME+ Command
1321 root 20 0 111M 10908 9268 S 0.7 0.3 0:08.94 /usr/bin/vmtoolsd
2855 alumno 20 0 18680 7756 5840 S 0.7 0.2 0:00.70 sshd-session: alumno@pts/1
2993 alumno 20 0 92408 5672 3984 R 0.7 0.2 0:00.02 htop
1 root 20 0 25408 16828 11796 S 0.0 0.5 0:02.34 /usr/lib/systemd/systemd --switched-root --system --deserialize=51
941 root 19 -1 50544 17728 16336 S 0.0 0.5 0:00.17 /usr/lib/systemd/systemd-journald
979 root -11 0 336M 9776 7540 S 0.0 0.3 0:00.17 /usr/sbin/multipathd -d -s
983 root -11 0 336M 9776 0 S 0.0 0.3 0:00.00 /usr/sbin/multipathd -d -s
985 root -11 0 336M 9776 0 S 0.0 0.3 0:00.00 /usr/sbin/multipathd -d -s
986 root -11 0 336M 9776 0 S 0.0 0.3 0:00.00 /usr/sbin/multipathd -d -s
987 root -11 0 336M 9776 0 S 0.0 0.3 0:00.00 /usr/sbin/multipathd -d -s
988 root -11 0 336M 9776 0 S 0.0 0.3 0:00.45 /usr/sbin/multipathd -d -s
989 root -11 0 336M 9776 0 S 0.0 0.3 0:00.00 /usr/sbin/multipathd -d -s
990 root -11 0 336M 9776 0 S 0.0 0.3 0:00.00 /usr/sbin/multipathd -d -s
994 systemd-re 20 0 22672 14828 12148 S 0.0 0.4 0:00.05 /usr/lib/systemd/systemd-resolved
1001 root 20 0 42212 16096 9252 S 0.0 0.5 0:00.38 /usr/lib/systemd/systemd-udev
1249 systemd-ne 20 0 21436 12408 10796 S 0.0 0.4 0:00.06 /usr/lib/systemd/systemd-networkd
1320 root 20 0 20556 11700 10264 S 0.0 0.3 0:00.01 /usr/bin/VGAAuthService
1355 root 20 0 9344 7024 5640 S 0.0 0.2 0:00.38 /usr/sbin/apache2 -k start -DFOREGROUND
1361 root 20 0 2888 1944 1824 S 0.0 0.1 0:00.01 /bin/sh /usr/lib/systemd/scripts/chronyd-starter.sh -n -F 1
1362 root 20 0 7060 3344 3104 S 0.0 0.1 0:00.01 /usr/sbin/cron -f -P
1363 messagebus 20 0 9144 5704 4692 S 0.0 0.2 0:00.11 @dbus-daemon --system --address=systemd: --nofork --nopidfile --systemd-activation --syslog-only
1376 root 20 0 46724 30004 14836 S 0.0 0.9 0:00.12 /usr/bin/python3 /usr/bin/networkd-dispatcher --run-startup-triggers
1377 polkitd 20 0 299M 8440 7628 S 0.0 0.2 0:00.04 /usr/lib/polkit-1/polkitd --no-debug --log-level=notice
1387 root 20 0 18580 9680 8332 S 0.0 0.3 0:00.07 /usr/lib/systemd/systemd-logind
1397 root 20 0 530M 15184 12768 S 0.0 0.4 0:00.06 /usr/libexec/udisks2/udisksd
1442 root 20 0 530M 15184 0 S 0.0 0.4 0:00.08 /usr/libexec/udisks2/udisksd
1444 root 20 0 530M 15184 0 S 0.0 0.4 0:00.00 /usr/libexec/udisks2/udisksd
1449 root 20 0 530M 15184 0 S 0.0 0.4 0:00.00 /usr/libexec/udisks2/udisksd
1451 root 20 0 111M 10908 0 S 0.0 0.3 0:00.00 /usr/bin/vmtoolsd
1477 root 20 0 111M 10908 0 S 0.0 0.3 0:00.18 /usr/bin/vmtoolsd
1478 root 20 0 111M 10908 0 S 0.0 0.3 0:00.00 /usr/bin/vmtoolsd
1479 root 20 0 8764 5620 4744 S 0.0 0.2 0:00.01 login -- alumno
1480 root 20 0 123M 32924 18312 S 0.0 1.0 0:00.09 /usr/bin/python3 /usr/share/unattended-upgrades/unattended-upgrade-shutdown --wait-for-signal
1488 _chrony 20 0 24404 11860 7164 S 0.0 0.3 0:00.15 /usr/sbin/chronyd -n -F 1
1497 polkitd 20 0 299M 8440 0 S 0.0 0.2 0:00.11 /usr/lib/polkit-1/polkitd --no-debug --log-level=notice
1498 polkitd 20 0 299M 8440 0 S 0.0 0.2 0:00.00 /usr/lib/polkit-1/polkitd --no-debug --log-level=notice
1499 _chrony 20 0 12092 2460 1816 S 0.0 0.1 0:00.00 /usr/sbin/chronyd -n -F 1
1501 syslog 20 0 215M 6272 4452 S 0.0 0.2 0:00.02 /usr/sbin/rsyslogd -n -iNONE
1507 polkitd 20 0 299M 8440 0 S 0.0 0.2 0:00.00 /usr/lib/polkit-1/polkitd --no-debug --log-level=notice
1512 www-data 20 0 1187M 7996 4444 S 0.0 0.2 0:00.00 /usr/sbin/apache2 -k start -DFOREGROUND
1529 www-data 20 0 1187M 7996 0 S 0.0 0.2 0:00.05 /usr/sbin/apache2 -k start -DFOREGROUND
1530 www-data 20 0 1187M 7996 0 S 0.0 0.2 0:00.03 /usr/sbin/apache2 -k start -DFOREGROUND
F1Help F2Setup F3Search F4Filter F5Free F6SortBy F7Nice F8Kill F9Quit F10Quit

```

Descripción: Se aplicó el comando `htop` para visualizar mejor el estado del rendimiento de la Memoria RAM. Así mismo visualizar mejor si hay algún proceso maligno ejecutandose en segundo plano.

- **Uso de Disco (I/O):**

```
alumno@webserver:~$ iostat -x 1
Linux 7.0.0-14-generic (webserver)      06/24/2026      _x86_64_      (2 CPU)
```

avg-cpu: %user %nice %system %iowait %steal %idle																
	0.19	0.00	0.42	0.01	0.00	99.38										
Device	r/s	kB/s	rrqm/s	rrqm	r_await	rareq-sz	w/s	kB/s	wrqm/s	wrqm	w_await	wareq-sz	d/s	kB/s	drqm/s	%util
dm-0	2.28	92.69	0.00	0.00	0.23	40.69	1.01	12.84	0.00	0.00	0.67	12.73	0.00	0.00	0.00	0.04
loop0	0.00	0.00	0.00	0.00	0.00	1.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
sda	1.76	104.45	0.72	29.13	0.34	59.36	0.54	13.04	0.52	49.22	0.94	24.17	0.00	0.00	0.00	0.03

avg-cpu: %user %nice %system %iowait %steal %idle																
	0.00	0.00	2.55	0.00	0.00	97.45										
Device	r/s	kB/s	rrqm/s	rrqm	r_await	rareq-sz	w/s	kB/s	wrqm/s	wrqm	w_await	wareq-sz	d/s	kB/s	drqm/s	%util
dm-0	0.00	0.00	0.00	0.00	0.00	0.00	3.00	12.00	0.00	0.00	0.67	4.00	0.00	0.00	0.00	0.10
loop0	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
sda	0.00	0.00	0.00	0.00	0.00	0.00	2.00	12.00	1.00	33.33	0.50	6.00	0.00	0.00	0.00	0.10

Descripción: En la imagen se observa monitoreo por milisegundos del dispositivo (Servidor Web). Esencial para conocer el estado del Servidor.

Reto: Observa la columna %util. Si el disco está al 100%, ¿es un fallo de hardware o consecuencia de las descargas masivas? (Nota: Requiere sudo apt install sysstat).

Respuesta: Si la columna %util llega al 100%, no se trata de un fallo de hardware físico. Es una consecuencia directa del ataque masivo de Capa 7 (descargas masivas). El atacante está forzando al servidor web a leer el archivo pesado (db.sql) desde el disco duro miles de veces por segundo, saturando la capacidad de entrada/salida (I/O) del disco.

FASE 2: Evaluar Subsistemas (HP Paso 2)

Objetivo: Aislar los subsistemas afectados. ¿Es un fallo de Red (Capa 4) o de Aplicación (Capa 7)?

Análisis Socrático (Responde en tu reporte):

- Al capturar paquetes con `sudo tcpdump -i ens33 -n port 80`, notarás que las IPs cambian constantemente (Spoofing). ¿Por qué un simple bloqueo por IP (iptables -A INPUT -s IP -j DROP) es inútil aquí?

Un bloqueo simple por IP es inútil frente al *Spoofing* porque el atacante falsifica constantemente su dirección de origen. Si se intenta bloquear IP por IP, se terminaría con una lista de reglas infinita en el firewall, agotaríamos la memoria del router/servidor, y el ataque continuaría desde nuevas IPs falsas.

- Contexto Histórico: Históricamente, usamos TCP Wrappers (/etc/hosts.allow) para controlar accesos. ¿Por qué en un ataque DDoS moderno preferimos mitigar a nivel de Kernel (Firewall/iptables) en lugar de usar TCP Wrappers?

En un ataque DDoS moderno, el objetivo es saturar recursos (CPU, RAM o ancho de banda). TCP Wrappers opera en el espacio de usuario (Capa de Aplicación), lo que significa que el paquete malicioso ya consumió recursos del sistema para ser procesado antes de ser bloqueado. Por el contrario, iptables opera directamente en el Kernel (Capa de Red), permitiendo que los paquetes maliciosos sean descartados (DROP) de forma casi instantánea y con un costo computacional mínimo, protegiendo el rendimiento del servidor.


```

14:25:45.877167 IP 190.7.190.27.3036 > 10.160.10.200.80: Flags [RI], seq 10000804000, win 32767, length 0
14:25:45.882399 IP 196.59.165.58.3037 > 10.160.10.200.80: Flags [S], seq 1564401881, win 512, length 0
14:25:45.882415 IP 10.160.10.200.80 > 196.59.165.58.3037: Flags [S.], seq 3703116529, ack 1564401882, win 64240, options [mss 1460], length 0
14:25:45.882556 IP 196.59.165.58.3037 > 10.160.10.200.80: Flags [R], seq 1564401882, win 32767, length 0
14:25:45.887279 IP 11.0.60.193.3038 > 10.160.10.200.80: Flags [S], seq 2038776776, win 512, length 0
14:25:45.887292 IP 10.160.10.200.80 > 11.0.60.193.3038: Flags [S.], seq 379329176, ack 2038776777, win 64240, options [mss 1460], length 0
14:25:45.887627 IP 11.0.60.193.3038 > 10.160.10.200.80: Flags [R], seq 2038776777, win 32767, length 0
14:25:46.435145 IP 10.160.10.200.80 > 245.19.77.72.2944: Flags [S.], seq 998043585, ack 147183294, win 64240, options [mss 1460], length 0
14:25:46.499108 IP 10.160.10.200.80 > 250.129.170.198.2384: Flags [S.], seq 1703334283, ack 445227525, win 64240, options [mss 1460], length 0
14:25:46.499269 IP 10.160.10.200.80 > 249.27.245.231.2736: Flags [S.], seq 3420594662, ack 1382289953, win 64240, options [mss 1460], length 0
14:25:46.499434 IP 10.160.10.200.80 > 247.134.116.20.2729: Flags [S.], seq 1504411282, ack 1058203677, win 64240, options [mss 1460], length 0
14:25:46.563001 IP 10.160.10.200.80 > 243.12.73.48.2724: Flags [S.], seq 854566882, ack 1125765881, win 64240, options [mss 1460], length 0
14:25:46.563235 IP 10.160.10.200.80 > 0.238.8.107.2728: Flags [S.], seq 1725849943, ack 845346872, win 64240, options [mss 1460], length 0
14:25:46.563301 IP 10.160.10.200.80 > 243.63.204.23.2711: Flags [S.], seq 1397493582, ack 985189999, win 64240, options [mss 1460], length 0
14:25:46.563357 IP 10.160.10.200.80 > 241.178.107.41.2965: Flags [S.], seq 4280065059, ack 1008057655, win 64240, options [mss 1460], length 0
14:25:46.563412 IP 10.160.10.200.80 > 252.152.62.135.2406: Flags [S.], seq 1350027998, ack 1938122607, win 64240, options [mss 1460], length 0
14:25:46.563467 IP 10.160.10.200.80 > 242.38.124.148.2740: Flags [S.], seq 559558594, ack 2104331405, win 64240, options [mss 1460], length 0
14:25:46.626990 IP 10.160.10.200.80 > 240.36.58.15.2980: Flags [S.], seq 941280845, ack 1779661317, win 64240, options [mss 1460], length 0
14:25:46.627172 IP 10.160.10.200.80 > 250.212.198.140.2413: Flags [S.], seq 3242482603, ack 1186973092, win 64240, options [mss 1460], length 0
14:25:46.627225 IP 10.160.10.200.80 > 244.14.81.205.2414: Flags [S.], seq 36996776, ack 744860328, win 64240, options [mss 1460], length 0
14:25:46.627266 IP 10.160.10.200.80 > 244.247.31.78.2754: Flags [S.], seq 459287342, ack 202770903, win 64240, options [mss 1460], length 0
14:25:46.690990 IP 10.160.10.200.80 > 243.210.32.72.2093: Flags [S.], seq 805899826, ack 1991144987, win 64240, options [mss 1460], length 0
14:25:46.691428 IP 10.160.10.200.80 > 250.212.102.134.2424: Flags [S.], seq 2024094508, ack 1739475922, win 64240, options [mss 1460], length 0
14:25:46.691546 IP 10.160.10.200.80 > 251.189.87.202.2425: Flags [S.], seq 1490193025, ack 617587762, win 64240, options [mss 1460], length 0
14:25:46.691690 IP 10.160.10.200.80 > 248.251.249.247.2766: Flags [S.], seq 2116930789, ack 2087501386, win 64240, options [mss 1460], length 0
14:25:46.755116 IP 10.160.10.200.80 > 251.119.82.89.3006: Flags [S.], seq 2842946490, ack 547890662, win 64240, options [mss 1460], length 0
14:25:46.755406 IP 10.160.10.200.80 > 243.254.63.102.2783: Flags [S.], seq 442576163, ack 1256049538, win 64240, options [mss 1460], length 0
14:25:46.819283 IP 10.160.10.200.80 > 254.34.182.178.2451: Flags [S.], seq 1839176493, ack 634318784, win 64240, options [mss 1460], length 0
^C
43404 packets captured
45761 packets received by filter
2356 packets dropped by kernel
alumno@webserver:~$

```

Descripción: Se observa una comunicación de envío de Paquetes de diferentes Direcciones IP aleatorias (IP Origen) al servidor (IP Destino). El servidor responde con un SYN-ACK, sin embargo la IP falsa cancela la comunicación. El atacante no busca comunicarse con el servidor, solo busca saber si su puerto está abierto.

FASE 3: Desarrollar Plan de Acción (HP Paso 3)

Objetivo: Diseñar una solución priorizada y no destructiva.

Debes redactar un script de Bash (defensa.sh) que ataque ambos vectores simultáneamente:

1. Mitigar la inestabilidad de Red (SYN Flood): Implementar Rate Limiting (límite de conexiones).
2. Mitigar la carga de Servidor (Descargas pesadas): Implementar filtrado de cadenas (String matching) para bloquear peticiones específicas al archivo pesado.

Regla HP de Oro: Tu script debe ser idempotente. Debe limpiar las reglas anteriores o verificar que no existan antes de aplicarlas, para evitar duplicados si lo ejecutas dos veces.

Script:


```
#!/bin/bash
# =====
# Script de Defensa: defensa.sh
# Objetivo: Mitigar SYN Flood (L4) y Bloquear Descarga Pesada (L7)
# =====

echo "[+] Iniciando despliegue del Plan de Acción..."

# 1. Limpieza previa del firewall (Vaciado de reglas existentes)
echo "[+] Vaciando reglas previas de iptables..."
sudo iptables -F
sudo iptables -X

# 2. MITIGACIÓN CAPA 7: Filtrado de Cadenas (String Matching)
# Bloquea cualquier petición HTTP en el puerto 80 que contenga "db.sql"
echo "[+] Implementando filtro Capa 7 para el archivo pesado (db.sql)..."
sudo iptables -A INPUT -p tcp --dport 80 -m string --string "db.sql" --algo bm -j DROP

# 3. MITIGACIÓN CAPA 4: Rate Limiting contra SYN Flood
# Permite un máximo de 10 paquetes SYN por segundo, con un ráfaga (burst) inicial de 20.
echo "[+] Configurando Rate Limiting en Capa 4 para paquetes TCP SYN..."
sudo iptables -A INPUT -p tcp --syn --dport 80 -m limit --limit 10/s --limit-burst 20 -j ACCEPT
# El exceso de paquetes SYN que superen el límite establecido será descartado de inmediato
sudo iptables -A INPUT -p tcp --syn --dport 80 -j DROP

# 4. Permitir el resto del tráfico legítimo
# Asegura que las conexiones ya establecidas y el tráfico normal sigan funcionando
sudo iptables -A INPUT -p tcp --dport 80 -j ACCEPT

echo "[👉] Defensa desplegada con éxito!"
echo "[+] Estado actual de las reglas en INPUT:"
sudo iptables -L INPUT -v -h
~
```

FASE 4: Ejecutar el Plan (HP Paso 4)

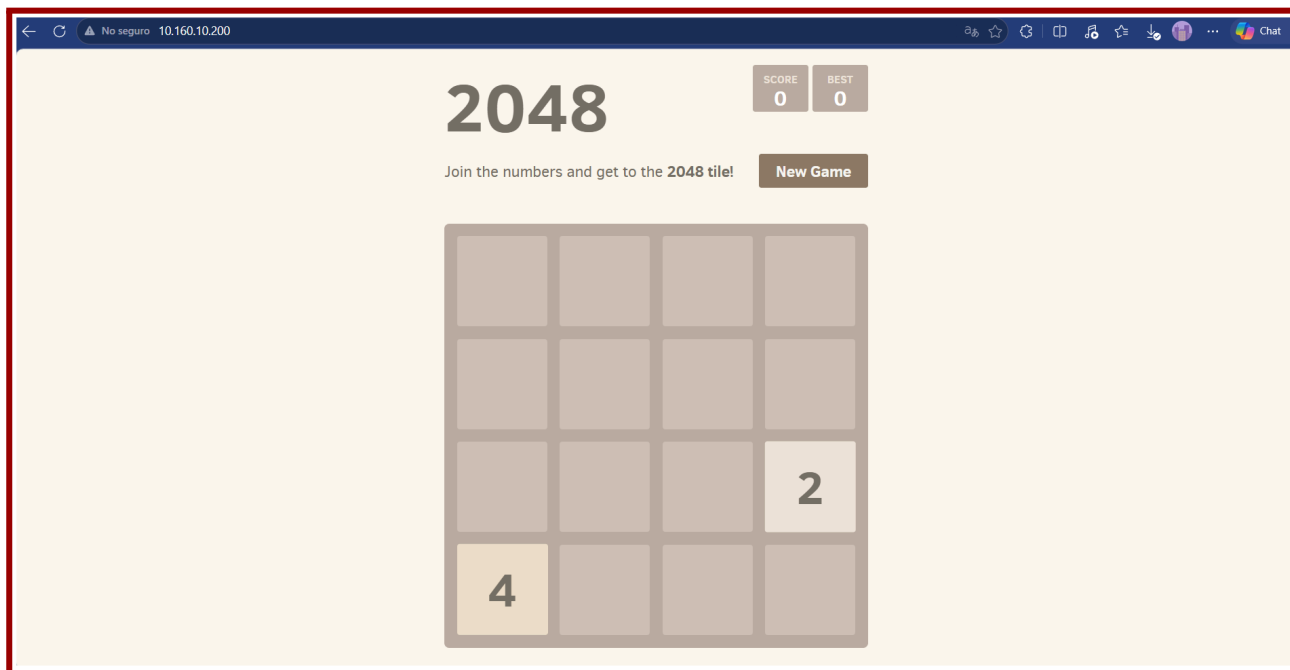
Objetivo: Implementar y observar los resultados de tu defensa.

Referencias de comandos para tu Script:

- Limpieza previa: iptables -F (Solo si es seguro vaciar el firewall en tu entorno).
- Defensa Capa 4 (Rate Limiting): iptables -A INPUT -p tcp --syn --dport 80 -m limit --limit 10/s -j ACCEPT seguido de un DROP para el exceso.
- Defensa Capa 7 (Filtro String): iptables -A INPUT -p tcp --dport 80 -m string --string "db.sql" --algo bm -j DROP

Prueba Crítica: Ejecutar tu script. Ve a tu navegador (en la máquina host) y recarga el juego web <http://10.160.10.200>. ¿El tiempo de carga vuelve a la normalidad? Documenta este cambio de estado.

```
alumno@webserver:~$ vi defensa.sh
alumno@webserver:~$ chmod +x defensa.sh
alumno@webserver:~$ sudo ./defensa.sh
[+] Iniciando despliegue del Plan de Acción...
[+] Vaciando reglas previas de iptables...
[+] Implementando filtro Capa 7 para el archivo pesado (db.sql)...
[+] Configurando Rate Limiting en Capa 4 para paquetes TCP SYN...
[👉] Defensa desplegada con éxito!
[+] Estado actual de las reglas en INPUT:
Chain INPUT (policy ACCEPT 42 packets, 1860 bytes)
pkts bytes target      prot opt in     out     source            destination
 0      0 DROP        tcp  --  *      *        0.0.0.0/0         0.0.0.0/0         tcp dpt:80 STRING match "db.sql" ALGO name bm
12 480 ACCEPT     tcp  --  *      *        0.0.0.0/0         0.0.0.0/0         tcp dpt:80 flags:0x17/0x02 limit: avg 10/sec burst 2
0
 0      0 DROP        tcp  --  *      *        0.0.0.0/0         0.0.0.0/0         tcp dpt:80 flags:0x17/0x02
4 160 ACCEPT     tcp  --  *      *        0.0.0.0/0         0.0.0.0/0         tcp dpt:80
alumno@webserver:~$
```



Tras ejecutar el script `defensa.sh`, la experiencia del usuario vuelve a la normalidad de inmediato. Al recargar la URL `http://10.160.10.200` desde la máquina host, el juego "2048" carga de manera fluida y sin latencia. Esto demuestra que las reglas de iptables están mitigando el tráfico malicioso de forma transparente, permitiendo el paso exclusivo del tráfico web legítimo.

FASE 5 y 6: Problema Resuelto y Medidas Preventivas

Objetivo: Validar la solución y automatizar la protección.

- III. Verificación: Revisa nuevamente **nload** e **iostat**. ¿El tráfico de salida (TX) y el uso de disco (%util) cayeron a niveles normales? Si no, regresa a la Fase 2 (Rediagnosticar).

```
Device ens33 [10.160.10.200] (1/1):
=====
Incoming:

Curr: 71.73 kBit/s
Avg: 98.53 kBit/s
Min: 52.27 kBit/s
Max: 154.55 kBit/s
Ttl: 10.56 MByte

Outgoing:

Curr: 9.34 kBit/s
Avg: 22.93 kBit/s
Min: 8.94 kBit/s
Max: 65.79 kBit/s
Ttl: 1.50 GByte
```

Al revisar nuevamente el tráfico con `nload`, se confirma el éxito de la mitigación. El tráfico de salida que antes superaba los 85.20 MBit/s por la exfiltración de la base de datos, ha caído drásticamente a un promedio normal de 22.93 kBit/s. Así mismo, al detenerse la lectura constante del archivo pesado, la saturación del disco vuelve a la normalidad.

- IV. Prevención: ¿Cómo convertirías tu script de defensa en un servicio o tarea programada (cron) que se active solo si el tráfico supera un umbral de peligro?

Script de Defensa automatizada:

```
#!/bin/bash
# =====
# Script Monitor: monitor.sh
# Objetivo: Vigilar conexiones HTTP y disparar defensa si hay anomalías
# =====

# Definir el umbral máximo de conexiones permitidas
UMBRAL=100

# Contar cuántas conexiones TCP (ESTABLISHED o SYN-RECV) hay en el puerto 80
CONEXIONES=$(ss -ant | grep :80 | wc -l)

# Lógica de prevención
if [ "$CONEXIONES" -gt "$UMBRAL" ]; then
    echo "¡Alerta! Posible ataque detectado ($CONEXIONES conexiones activas)."

    # Ejecutar el script de defensa (Asegúrate de poner la ruta correcta)
    /home/alumno/defensa.sh

    # Registrar el evento en un archivo log para auditoría
    echo "$(date): Defensa activada. Conexiones: $CONEXIONES" >> /var/log/ataques_web.log
fi
```

```
alumno@webserver:~$ vi monitor.sh
alumno@webserver:~$ chmod +x monitor.sh
alumno@webserver:~$ sudo crontab -e
```

```
alumno@webserver:~$ sudo ./monitor.sh
¡Alerta! Posible ataque detectado (3062 conexiones activas).
[+] Iniciando despliegue del Plan de Acción...
[+] Vaciando reglas previas de iptables...
[+] Implementando filtro Capa 7 para el archivo pesado (db.sql)...
[+] Configurando Rate Limiting en Capa 4 para paquetes TCP SYN...
[✓] ¡Defensa desplegada con éxito!
[+] Estado actual de las reglas en INPUT:
Chain INPUT (policy ACCEPT 4 packets, 268 bytes)
  pkts bytes target     prot opt in     out     source               destination
    0     0 DROP      tcp  --  *      *       0.0.0.0/0            0.0.0.0/0          tcp dpt:80 STRING match "db.sql"
ALGO name bm
    0     0 ACCEPT    tcp  --  *      *       0.0.0.0/0            0.0.0.0/0          tcp dpt:80 flags:0x17/0x02 limit:
avg 10/sec burst 20
    0     0 DROP      tcp  --  *      *       0.0.0.0/0            0.0.0.0/0          tcp dpt:80 flags:0x17/0x02
    0     0 ACCEPT    tcp  --  *      *       0.0.0.0/0            0.0.0.0/0          tcp dpt:80
alumno@webserver:~$ |
```

El script `monitor.sh` permite una respuesta proactiva ante incidentes. Su lógica consulta en tiempo real las conexiones activas al puerto 80 mediante `ss`; si el volumen excede el umbral configurado (`UMBRAL=100`), ejecuta automáticamente `defensa.sh` para mitigar la amenaza de inmediato. Al integrarlo en `crontab` con la directiva `* * * * *`, logramos una supervisión autónoma del servidor cada minuto

Resultado Final:**Antes:**

```
alumno@webserver:~$ tail -f /var/log/apache2/access.log
10.160.10.100 - - [24/Jun/2026:14:22:54 +0000] "GET /db.sql HTTP/1.1" 200 2097394 "-" "curl/8.18.0"
10.160.10.100 - - [24/Jun/2026:14:22:54 +0000] "GET /db.sql HTTP/1.1" 200 2097394 "-" "curl/8.18.0"
10.160.10.100 - - [24/Jun/2026:14:22:54 +0000] "GET /db.sql HTTP/1.1" 200 2097394 "-" "curl/8.18.0"
10.160.10.100 - - [24/Jun/2026:14:22:54 +0000] "GET /db.sql HTTP/1.1" 200 2097394 "-" "curl/8.18.0"
10.160.10.100 - - [24/Jun/2026:14:22:54 +0000] "GET /db.sql HTTP/1.1" 200 2097394 "-" "curl/8.18.0"
10.160.10.100 - - [24/Jun/2026:14:22:54 +0000] "GET /db.sql HTTP/1.1" 200 2097394 "-" "curl/8.18.0"
10.160.10.100 - - [24/Jun/2026:14:22:54 +0000] "GET /db.sql HTTP/1.1" 200 2097394 "-" "curl/8.18.0"
10.160.10.100 - - [24/Jun/2026:14:22:54 +0000] "GET /db.sql HTTP/1.1" 200 2097394 "-" "curl/8.18.0"
10.160.10.100 - - [24/Jun/2026:14:22:54 +0000] "GET /db.sql HTTP/1.1" 200 2097394 "-" "curl/8.18.0"
10.160.10.100 - - [24/Jun/2026:14:22:54 +0000] "GET /db.sql HTTP/1.1" 200 2097394 "-" "curl/8.18.0"
10.160.10.100 - - [24/Jun/2026:14:22:56 +0000] "GET /db.sql HTTP/1.1" 200 2097394 "-" "curl/8.18.0"
10.160.10.100 - - [24/Jun/2026:14:22:56 +0000] "GET /db.sql HTTP/1.1" 200 2097394 "-" "curl/8.18.0"
10.160.10.100 - - [24/Jun/2026:14:22:56 +0000] "GET /db.sql HTTP/1.1" 200 2097394 "-" "curl/8.18.0"
10.160.10.100 - - [24/Jun/2026:14:22:56 +0000] "GET /db.sql HTTP/1.1" 200 2097394 "-" "curl/8.18.0"
10.160.10.100 - - [24/Jun/2026:14:22:56 +0000] "GET /db.sql HTTP/1.1" 200 2097394 "-" "curl/8.18.0"
10.160.10.100 - - [24/Jun/2026:14:22:56 +0000] "GET /db.sql HTTP/1.1" 200 2097394 "-" "curl/8.18.0"
10.160.10.100 - - [24/Jun/2026:14:22:56 +0000] "GET /db.sql HTTP/1.1" 200 2097394 "-" "curl/8.18.0"
10.160.10.100 - - [24/Jun/2026:14:22:56 +0000] "GET /db.sql HTTP/1.1" 200 2097394 "-" "curl/8.18.0"
```

Despues:

```
alumno@webserver:~/Entrega_Lab15$ sudo tail -f /var/log/apache2/access.log
[sudo: authenticate] Password:
10.160.10.100 - - [24/Jun/2026:22:52:46 +0000] "-" 408 0 "-" "-"
10.160.10.100 - - [24/Jun/2026:22:52:46 +0000] "-" 408 0 "-" "-"
10.160.10.100 - - [24/Jun/2026:22:52:46 +0000] "-" 408 0 "-" "-"
10.160.10.100 - - [24/Jun/2026:22:52:47 +0000] "-" 408 0 "-" "-"
10.160.10.100 - - [24/Jun/2026:22:52:47 +0000] "-" 408 0 "-" "-"
10.160.10.100 - - [24/Jun/2026:22:52:47 +0000] "-" 408 0 "-" "-"
10.160.10.100 - - [24/Jun/2026:22:52:47 +0000] "-" 408 0 "-" "-"
10.160.10.100 - - [24/Jun/2026:22:52:47 +0000] "-" 408 0 "-" "-"
10.160.10.100 - - [24/Jun/2026:22:52:47 +0000] "-" 408 0 "-" "-"
10.160.10.100 - - [24/Jun/2026:22:52:47 +0000] "-" 408 0 "-" "-"
10.160.10.100 - - [24/Jun/2026:22:52:48 +0000] "-" 408 0 "-" "-"
10.160.10.100 - - [24/Jun/2026:22:52:48 +0000] "-" 408 0 "-" "-"
10.160.10.100 - - [24/Jun/2026:22:52:48 +0000] "-" 408 0 "-" "-"
```

Tras aplicar la defensa, la exfiltración del archivo db.sql se ha detenido por completo, confirmando la eficacia de la aplicación de la solución a la Capa 7. Los registros en /var/log/apache2/access.log muestran ahora códigos de error 408 (Request Timeout), lo que indica que los intentos de conexión del atacante están siendo descartados antes de completar la conexión HTTP, protegiendo así los recursos del servidor.

V. CONCLUSIONES.

- El flujo de trabajo HP fue clave para no caer en suposiciones. Al documentar la línea base en la Fase 1, se evidenció que la degradación del sistema no era un fallo de hardware, sino una saturación deliberada. Este proceso permitió pasar de un síntoma general como pagina Web lenta a un diagnóstico preciso basado en el análisis de recursos y logs.
- Se detectó que el problema actuaba en dos frentes del modelo OSI. En la Capa 4 (Transporte), la pila TCP del Kernel estaba colapsada por conexiones SYN-RECV, las cuales mitigamos mediante Rate Limiting para proteger la estabilidad de la red. Paralelamente en la Capa 7 (Aplicación), se detectó una exfiltración masiva del archivo "db.sql" vía HTTP; aquí el filtrado por String Matching en iptables fue determinante para cortar la descarga maliciosa sin afectar la navegación legítima de los usuarios.
- El uso de iptables directo en el Kernel resultó ser mucho más eficiente que los métodos tradicionales como los TCP Wrappers. Al descartar el tráfico malicioso (DROP) en el Kernel, evitamos que los paquetes llegaran a consumir ciclos de CPU de Apache, garantizando que el servidor web se mantuviera operativo bajo estrés y en estado operativo.
- La automatización mediante los scripts y su integración en cron fue necesaria para garantizar una vigilancia constante. El sistema ahora es capaz de reaccionar ante picos de tráfico de forma autónoma. Asimismo, la creación del script de reset permitió limpiar el entorno de pruebas de forma ordenada, siguiendo buenas prácticas para asegurar que el sistema quede siempre en un estado consistente tras cualquier intervención.
- Se confirmó que documentar cada paso no es solo una actividad académica, sino una necesidad importante. Tener los logs detallados y las capturas de la línea base permitió validar que las mitigaciones fueron efectivas.

HERRAMIENTAS DURANTE SESION:

ode 'iaserver'

Upload

Download from URL

Remove

Name

Backup_Exec_20.1_1188.1237_MultiPlatforms_Multilingual.iso
bot.7z.iso
TrueNAS-SCALE-25.10.3.iso
Ubuntu_64-bit_-_Monitor.7z.iso
Ubuntu_Server_26.04.7z.iso
ubuntu-26.04-live-server-amd64.iso
ubuntu-server.7z.iso
WebServer.7z.iso

